CS 165 COMPUTER SECURITY • UC Riverside • Fall 2026 • **MIDTERM**

Name: _____ SID: _____

Instructions

- Closed book. No notes, calculators, phones, or other devices.
- **Six questions, each worth 20 points. Answer any FIVE**, for a maximum of 100.
- Tick the five you want marked in the box below. If you tick more than five, the first five answers will be considered.
- Answer in the space provided; continue on the back of the page if you need to.
- Short answers are expected — a few sentences, or a labeled sketch, is enough.
- Points are shown for each part and are a good guide to how much to write.
- Write as clearly as possible; if I can't understand your writing, you will lose points.
- Always (concisely) justify your answers.

Mark these five questions tick up to five

☐ **1** ☐ **2** ☐ **3** ☐ **4** ☐ **5** ☐ **6**

If you tick nothing, the first five questions you attempt will be marked.

1 Threat models and attack surface — 20 points

A public server runs `photod`, a thumbnail service that (1) reads images uploaded by any user, (2) reads a watermark file from a path only `root` can write, and (3) writes finished thumbnails into a web directory. Its image parser contains a known out-of-bounds write.

(a) [3 pts] Name the three properties of the **CIA triad**.

(b) [7 pts] Recall that a vulnerability is a flaw that is *accessible* to an adversary who can *exploit* it. For an adversary on the Internet, is the parser bug a vulnerability when reached by the path in (1)? What about when reached by path (2)? Justify each answer in terms of accessibility.

(c) [4 pts] Recall that the *attack surface* is the set of system calls a program makes that access adversary-modifiable resources. Which of (1)–(3) are on it here?

(d) [6 pts] The Morris worm (1988) hijacked `fingerd` through a buffer overflow and copied itself from host to host, reinfecting machines many times over. It stole no data and modified none. **(i)** Which of confidentiality, integrity and availability did it mainly violate, and why? **(ii)** What changed in commercial systems as a result?

2 Stack overflow — 20 points

This 32-bit function is compiled **without** a stack protector, and `req` and `len` are attacker-controlled:

```
void handle(char *req, int len) {
    int authed = 0;
    char buf[64];
    memcpy(buf, req, len);
    if (authed) grant_access();
}
```

Assume the compiler places `authed` immediately above `buf`, then the saved frame pointer (4 bytes), then the saved return address (4 bytes).

(a) [3 pts] In one short sentence, what is the main idea of a **buffer overflow**?

(b) [3 pts] In which direction does `memcpy` write relative to `buf`, and in what order does a growing overflow reach the other three items?

(c) [5 pts] Show your arithmetic for each: **(i)** how many bytes of filler precede the saved return address, and **(ii)** the smallest `len` that overwrites its first byte.

(d) [6 pts] The attacker chooses how many bytes to send. Describe two *different* attacks this one bug allows, and say what the attacker achieves in each. Does DEP/NX prevent the *overwrite*? If it does, then how? If it does not, then why not?

(e) [3 pts] Recall that memory errors are classified as *spatial*, *temporal* or *type*. Which kind is the bug in `handle()`, and why? Give one example each of the other two kinds.

3 Return-oriented programming — 20 points

NX stops shellcode, and often no single library function does what you want. ROP chains short sequences ("gadgets") that each end in `ret`. Suppose you have found:

```
g1:  pop eax ; ret
g2:  pop ebx ; ret
g3:  mov [ebx], eax ; ret
```

(a) [4 pts] What does **ROP** stand for, and what instruction does every gadget end in?

(b) [8 pts] Lay out the stack (the sequence of values the overflow writes) to store value `V` at address `ADDR` using these gadgets, and say what each `ret` does.

(c) [3 pts] Why does NX not stop ROP?

(d) [5 pts] What does ASLR force the attacker to do first?

4 Shellcode — 20 points

Instead of reusing existing code, the attacker writes their own machine instructions into the buffer and returns into them.

(a) [4 pts] What is **shellcode**?

(b) [6 pts] The same bytes are written as data and then executed as code. Explain how that is possible, and state the one assumption about the stack it depends on.

(c) [5 pts] Shellcode delivered through a string copy such as `strcpy` must contain no NUL (`0x00`) bytes. Why? Give the standard trick for zeroing a register without introducing one, and name one delivery path where the constraint does *not* apply.

(d) [5 pts] The classic payload makes a single system call. Which one, and how are the call number and its arguments passed on 32-bit Linux?

5 Attacking the heap — 20 points

(a) [4 pts] What does `free()` do, and what is a **dangling pointer**?

(b) [9 pts] An attacker overflows the chunk before chunk `P` and so controls `P`'s `fd` (forward pointer) and `bk` (backward pointer). (i) What does the unlink performed by `free(P)` hand them? (ii) Name *two distinct* targets they could overwrite with it, and say what each one gives them. (iii) State the check "safe unlinking" adds, and why forged pointers fail it.

(c) [7 pts] A use-after-free leaves a stale pointer `b`. Explain why such a bug is often *not* exploitable as it stands, and what the attacker does to make it reliable.

6 Defenses — 20 points

(a) [3 pts] Name the three *classic* defenses covered in this course.

(b) [4 pts] Give *one* way a stack canary is bypassed.

(c) [7 pts] Recall that control hijacking has *two targets*. Name each, name the defense for it, and give one line on how that defense works.

(d) [6 pts] Intel CET enforces only *coarse* CFI on the forward edge. Explain why an attacker can still find a usable target inside the coarse allowed set, and state what fine CFI would additionally check. Does that guarantee the target is excluded?